THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Chosen Software: VLC Media Player | License: GPL v2

Student Name	Addepalli Dhanush
Registration Number	24MEI10159
Slot	A13
Date of Submission	31 March 2026
Course	*Open Source Software* | *OSS NGMC*

Part A — Origin and Philosophy

A1. The Problem This Software Was Created to Solve

Prior to the development and availability of the VLC media player, it was surprisingly frustrating to attempt to play video or stream media on computers. In the late 1990s, most media players were very limited and proprietary and relied heavily on the codec that was used. For instance, if one were to attempt to play a video they had downloaded and the media player they were using did not recognize the codec of the video file they were attempting to open, the video simply would not open. In addition to this problem, if one were attempting to stream video over the network, there were virtually no free tools that could stream video across the network. In the late 1990s, most universities were in need of this capability because different systems and formats needed to be compatible with one another.

The development of the VLC media player came out of the need for students at the École Centrale Paris to be able to stream video across the campus network. However, the software that was available at the time was either too expensive, too rigid and proprietary, or was simply not open-source enough to be altered to meet the needs of the students.

What made VLC unique wasn't just its functionality but also its philosophy. The project, which would later be named VideoLAN Project, was made open-source. Rather than making it proprietary or commercializing it too quickly, it was put at the disposal of the world. This enabled other developers around the world to work on it, make it more compatible with more codecs, and make it more reliable. Eventually, VLC would become synonymous with "playing anything," which is exactly what was needed following the chaos that had been caused by all of these formats and ecosystems.

VLC, in short, was able to address the fundamental issue of accessibility in digital media. It eliminated barriers. Users no longer had to think about what format something was in, what codecs were missing, or whether or not they had access to certain software. It simply worked. It was cross-platform, it was reliable, and it was liberating. What started as a project created by students to fix an issue on their own campus would go on to become one of the most popular media players in the world because it addressed a real-world problem.

A2. The License — What It Actually Says

The Four Freedoms of Free Software

The VLC media player software is covered by the GNU General Public License, which is a commonly used free software license for software packages. It was developed by Richard Stallman, who is the founder of the Free Software Foundation. It is intended to assure users of their basic freedom in using the software. These freedoms include four main freedoms: the freedom to use the software for any purpose, the freedom to study how the software works, the freedom to modify the software to meet their needs, and the freedom to distribute the software in both its original form and in modified form. All of these freedoms are granted by the VLC media player software, which qualifies it as free software.

Can a company take this software, modify it, and sell it without sharing their changes?

One of the key features of the GNU GPL is that it provides something called “copyleft,” which affects how modified versions of the software can be distributed. A company is at liberty to make use of VLC, modify it if they wish, and even sell it. However, they cannot make any money from selling any modified versions of the software. Moreover, they cannot distribute any modified versions of the software in any way that is proprietary. When they make any distributions of modified versions of the software, they have to

make sure that access is provided to the source code, and that all distributions are made using the same license.

GPL vs MIT — Differences and Your Choice

This is in stark contrast to other, more liberal licenses like the MIT license. The GPL requires that if modifications are made to software, then it too has to be open-source. However, in the case of the MIT license, programmers are allowed to make use of, modify, and distribute software as part of other closed-source applications without having to distribute the source code. This is due to fundamentally different ideologies. One requires that users be allowed to make use of software in any version, whereas the other requires that software be flexible and widely used, even if it means it is not open-source. For a software like VLC that was developed around the idea of collaboration and openness, this is important.

Free as in Beer vs Free as in Freedom

Lastly, it is also important to understand that there is a difference between “free as in free beer” and “free as in freedom.” The former means that it is available for free, whereas the latter means that it is available for users to make modifications and distribute. VLC is both. Not only is it available for free, but it is also available for users to make modifications and distribute. This is true even if it is part of other applications that are for sale. This is due to the rights that users have.

A3. The Ethics of Open Source

Should All Software Be Open Source?

The ideal of open source software is certainly appealing. However, the question of whether all software should be open source is complex. On one side, open source software is the ideal way to promote openness and collaboration. It provides the ability for all individuals to view and learn from the code. This is particularly important in critical

sectors such as education and security. However, it is important to recognize that not all software can be open source. In some software development scenarios, the data that is used is sensitive and proprietary. In such scenarios, the promotion of open source software might be detrimental. Companies that invest heavily in the development of software for specialized purposes might do so in the context of protecting the intellectual property. Therefore, it is important to recognize that the ideal of open source software is important. However, the reality is that the promotion of open source and closed source software is more important.

Is It Ethical to Profit From Free Community Labor?

The ethics of making money on top of an open-source software base can also be complex. Companies such as Red Hat, Google, and Meta use open-source software such as Linux as a base, which leads to the question of whether it is fair to make money from something that was developed by an open-source community. The justification for this is that, in most cases, companies such as Red Hat, which make their living from open-source software, contribute back to the community by developing the software. This means that they are giving back to the community that gives them their livelihood, which can be justified on an ethical level. The issue arises, however, when companies make use of the community but do not contribute back, thus benefiting from free community work.

Standing on the Shoulders of Giants

The phrase “a man can stand on the shoulders of giants,” which was originally used by Sir Isaac Newton to describe his use of the work of Galileo Galilei, can be used to describe the use of innovation by making use of the work of those who came before us. Open-source software can be used to describe this, as it allows those who use it to make use of the work of those who came before them, thus making use of the phrase to describe itself. It can be argued that this diminishes innovation, as people are making use of something that was originally created by someone else. However, in most cases, this innovation is what leads to new innovation, as most new innovation is a

> combination of old innovation, thus making the use of innovation by making use of something that was created by an open-source community an increase in innovation, rather than a decrease.

Part B — Linux Footprint on a Linux System

Installation Method

The VLC media player integrates into a Linux system in a relatively light and user-friendly manner, as expected from a media player designed as a desktop application and not as a service. VLC media player installation on a Linux system, such as Ubuntu or Debian, typically involves the use of the package manager (APT), and this can be done by simply invoking the command `sudo apt install vlc`. It can also be installed on RPM-based distributions, such as Fedora, by invoking `dnf install vlc`, and sometimes even as a Snap or Flatpak package. However, for a standard user, there is no need to manually compile the package from source. The package manager will take care of all the dependencies and will install the media player in accordance with the standard directory structure of the system.

Key Directories

VLC media player's components are installed on a Linux system, and all the components are installed in standard locations. The executable of the media player, `vlc`, can be found in `/usr/bin/vlc`, allowing it to be invoked from any location on the system. The plugins and libraries required for the media player are stored in directories such as `/usr/lib/vlc/` or `/usr/lib64/vlc/`, depending on the architecture of the system. The configuration files of the media player are not stored in any particular location, but a separate configuration directory for each user can be found in `~/.config/vlc/`. The media player does not create any significant log files in `/var/log/`, as it is not a service and does not require any significant interactions.

User and Group — Security Context

VLC does not operate as a separate user or group, unlike server software. It simply runs as the user that executes it. This is a significant security practice, as VLC works with media files that are not necessarily trusted. Running it as a non-privileged user

helps prevent damage if a vulnerability in VLC is exploited. In this way, the security of VLC follows the principle of least privilege, a fundamental principle of Linux security.

Service Management Commands

VLC does not operate as a service, managed by the system's service manager or init system, as it does not need to operate in the background. It can be started and stopped by the user through the command line or a GUI. However, for more experienced users, VLC can be operated in a scripted fashion or in "headless mode" for streaming, and it can then be managed as a service. In this way, a user can define a service that can be started or stopped with `systemctl --user start vlc` or `systemctl --user stop vlc`, though this is not part of a standard VLC installation.

Update Model — From Patch to Your Machine

VLC's updates are managed through the package management system of the Linux operating system. When a new version of VLC or a security patch for it is released by the VideoLAN Project, the package maintainers for the operating system package up the update and distribute it through official package repositories. Users of the operating system can then update VLC through standard update tools such as `sudo apt update && sudo apt upgrade`. In this way, security patches are distributed in a way that is managed and verified, and VLC works as part of the larger system of operating system updates.

Part C — The FOSS Ecosystem

What other open-source tools does it depend on?

VLC player is dependent on a wide range of open-source libraries and system components to provide its 'play anything' feature. VLC player is based on multimedia frameworks such as FFmpeg, which is used to encode and decode different audio and video formats. VLC player also uses libraries such as libdvdcss, which is used to play DVDs with CSS encryption, and X11 or Wayland, which is used to render graphical output in Linux systems. VLC player also uses different components of the GNU toolchain, which include compilers such as GCC and autotools, which is used during the compilation of VLC player code. The different components of VLC player show that VLC player is not an independent creation but is based on the integration of different open-source technologies.

What has it enabled or inspired?

VLC player has also enabled and inspired different projects and use cases in the open-source ecosystem. VLC player is used as a platform to develop custom media player applications, IPTV applications, and different types of media player applications. VLC player is not forked often to develop different and independent applications, but its underlying framework, libVLC, is used by different developers to develop their own applications with different types of media player capabilities. VLC player is used to develop different types of mobile applications, and different types of applications related to surveillance and broadcasting also use VLC player components.

Where does it fit in the LAMP stack, or how does it relate to the web?

VLC is not directly related to the LAMP stack, since it is not a web server or database-driven system. Nevertheless, VLC is related to the web and network ecosystem, where streaming protocols are implemented, and VLC supports protocols such as HTTP, RTP, and RTSP. This enables VLC to be both a client and a streaming

server. As such, VLC is related to the web ecosystem, where the VLC media player is utilized to test streaming protocols or act as a bridge between the source and the web-based system. Therefore, VLC is related to the web ecosystem but not through the backend system; VLC is related to the web ecosystem through its ability to handle multimedia data.

Is there a community

The VLC project is supported through the VideoLAN project, which is a non-profit organization that has been established globally. This indicates that the VLC project is supported globally, which is a key characteristic of the VLC project. This is due to the fact that the VLC project is supported through the VideoLAN project, which is a non-profit organization that has been established globally. This is due to the fact that the VLC project is supported through the VideoLAN project, which is a non-profit organization that has been established globally.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparison Table: Linux Kernel vs Windows / macOS

Dimension	Open Source (VLC Media Player)	Proprietary Alternative (Windows Media Player)
Cost	Completely free to download, use, and distribute with no licensing fees.	Free with Windows OS, but tied to the operating system; not independently distributed.
Security (code audit)	Source code is publicly available; anyone can inspect, audit, and identify vulnerabilities.	Source code is closed; only the company (Microsoft) can audit and fix issues internally.
Support & reliability	Community-driven support via forums and documentation; reliable but depends on community contributions.	Official support from Microsoft; structured updates and customer support, often more predictable for enterprise use.
Freedom to modify	Full freedom to modify, redistribute, and customize under GPL license.	No access to source code; users cannot modify or redistribute the software.
Community vs corporate control	Governed by the VideoLAN Project with global contributors; decisions are community-oriented.	Fully controlled by Microsoft; development direction depends on corporate priorities.
Overall verdict	Highly flexible, transparent, and independent; ideal for users who value control, compatibility, and openness.	More integrated and polished within its ecosystem; suitable for users who prefer simplicity and official support over flexibility.

Shell Script Tasks

Script 1 — System Identity Report

Concepts: variables, echo, command substitution $(), basic output formatting.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Addepalli Dhanush | Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Addepalli Dhanush"
SOFTWARE_CHOICE="Linux"

# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATE_TIME=$(date)
DISTRO=$(grep '^PRETTY_NAME' /etc/os-release | cut -d= -f2 | tr -d '"')

# --- Display ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software        : $SOFTWARE_CHOICE"
echo "Distribution    : $DISTRO"
echo "Kernel Version  : $KERNEL"
echo "User            : $USER_NAME"
echo "Home Directory  : $HOME_DIR"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE_TIME"
echo "----------------------------------------"
echo "License Info    : Linux is distributed under the GNU General Public License
(GPL)."
echo "========================================"
```

This Bash script acts like a simple “welcome screen” for a Linux system by displaying important system details in a clear, formatted way. It collects information such as the Linux distribution name, kernel version, current user, home directory, system uptime, and current date/time. The script uses **variables** to store these values (e.g., kernel, user, uptime), making the code organized and reusable. It uses **command substitution** `$( )` to execute system commands like `uname`, `whoami`, `uptime`, and `date`, and capture their outputs. The **`echo` command** is used to print all the information to the terminal. For better

> readability, **output formatting** is applied using separator lines made of `=` and `-`, which give the output a structured, welcome-screen appearance.

```
┌──(aryan㉿kali)-[~]
└─$ nano system_report.sh

┌──(aryan㉿kali)-[~]
└─$ ./system_report.sh
========================================
 Open Source Audit – Addepalli Dhanush
========================================
Software        : Linux
Distribution    : Kali GNU/Linux Rolling
Kernel Version  : 6.18.12+kali-arm64
User            : aryan
Home Directory  : /home/aryan
Uptime          : up 5 minutes
Date & Time     : Tue Mar 31 12:54:50 AM IST 2026
----------------------------------------
License Info    : Linux is distributed under the GNU General Public License (GPL).
========================================

┌──(aryan㉿kali)-[~]
└─$
```

Script 2 — FOSS Package Inspector

Concepts: if-then-else, case statement, dpkg / rpm, pipe with grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Addepalli Dhanush

PACKAGE="vlc"   # Change this if needed

# Check if package is installed (works for Debian/Ubuntu systems)
if dpkg -l | grep -w $PACKAGE &>/dev/null; then
    echo "$PACKAGE is installed."
    dpkg -l | grep -w $PACKAGE
else
    echo "$PACKAGE is NOT installed."
fi

echo "----------------------------------------"

# Case statement for package description
case $PACKAGE in
    httpd) echo "Apache: the web server that built the open internet" ;;
    mysql) echo "MySQL: open source at the heart of millions of apps" ;;
    vlc) echo "VLC: a powerful open-source media player for all formats" ;;
    firefox) echo "Firefox: privacy-focused open-source web browser" ;;
    gimp) echo "GIMP: free and open-source image editing software" ;;
    *) echo "Unknown package: no description available" ;;
```

```
esac
```

This script essentially acts as a package inspector that inspects whether or not a given software package is installed. It then proceeds to print relevant information about it, along with a small description.

The if-then-else block essentially inspects whether or not the package is installed. It uses a command that is similar to dpkg -s $PACKAGE (dpkg -l | grep), which essentially inspects the package database. If the command is successful (exit value 0), then it is assumed that the package is installed, and relevant information is displayed. If not, then the else part is executed, which states that the package is not installed.

The dpkg -s command is used here to obtain detailed status information about a package. This includes its version, description, etc. When we redirect this to grep -E 'Version|License|Summary', it essentially searches for lines that have any of these keywords. This is where grep -E (extended grep) is used.

The case statement is then used, which essentially branches depending on what $PACKAGE is. The patterns that we have defined here, such as httpd, mysql, vlc, etc., essentially match package names. We then print a small description of what that particular software is. If none of these match, then we have the default case.

Finally, we have &>/dev/null, which essentially suppresses both standard output and error messages that might be produced by the command. This is used so that unwanted messages, such as “package not found,” do not get displayed on the screen.

```
┌──(aryan㉿kali)-[~]
└─$ ./package_inspector.sh
vlc is installed.
ii  vlc-bin                                3.0.23-1                          arm64        binaries from VLC
-----------------------------------
VLC: a powerful open-source media player for all formats

┌──(aryan㉿kali)-[~]
```

Script 3 — Disk and Permission Auditor

Concepts: for loop, arrays, du, ls -ld, awk, cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Addepalli Dhanush

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------------------------"
echo "Software Config Directory Check"
echo "----------------------------------------"

# Change this according to your chosen software
CONFIG_DIR="/etc/vlc"

if [ -d "$CONFIG_DIR" ]; then
    CONFIG_INFO=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
    echo "$CONFIG_DIR exists => Permissions: $CONFIG_INFO"
else
    echo "$CONFIG_DIR does NOT exist"
fi
```

This script checks and audits important system directories. The script does this by going through a set of directories defined in an array and then printing information about these directories.

The **for loop** statement is defined as `for DIR in "${DIRS[@]}"`, which means that it will iterate over each directory in the array. The array will be expanded in the for loop, and it will go through each directory separately (for example, `/etc`, `/var/log`, etc.).

Inside the for loop, the **`if [ -d "$DIR" ]` condition** is checked. This condition checks if the directory exists. The `-d` flag is used to ensure that only directories are checked and not files. This is important to prevent errors in case the directory does not exist.

Next, the **`ls -ld $DIR` command** is executed. This will display information about the directory itself and not its contents. The information displayed includes directory permissions, owner ID, and group ID. The information is then passed through **`awk '{print $1, $3, $4}'`,** which means that only three fields will be displayed. These fields are permission bits, owner ID, and group ID.

Next, the **`du -sh $DIR` command** is executed. This will display information about the total disk space used in the directory. The `-s` flag is used to display only total sizes, and `-h` is used to display sizes in human-readable format (KB, MB, etc.). The information is then passed through **`cut -f1`,** which means that only the first column will be displayed.

Lastly, **`2>/dev/null` is used to suppress error messages. This is important to prevent error messages like “Permission denied” from appearing in case we try to execute this script in restricted directories.

Script 4 — Log File Analyzer

Concepts: while read loop, if-then, counter variables, command-line arguments ($1, $2).

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Addepalli Dhanush
# Usage: ./log_analyzer.sh /var/log/messages error

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error'
COUNT=0

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Do-while style retry if file is empty
while [ ! -s "$LOGFILE" ]; do
    echo "File is empty. Waiting for content..."
    sleep 2
done

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "----------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

```
echo "----------------------------------------"

# Show last 5 matching lines
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

This script reads a log file using the provided inputs, which are read at runtime, and processes the file line by line to find keyword matches.

The variables **`$1` and `$2`** are **command-line arguments**. When you run the script with the following command:

```bash
./log_analyzer.sh sample.log error
```

* `$1` becomes **`sample.log`** (log file)
* `$2` becomes **`error`** (keyword to search for)

The syntax for **`${2:-error}`** means: “use `$2` if provided, otherwise use `error`.” So, if you run the script with the following command:

```bash
./log_analyzer.sh sample.log
```

The script will automatically look for the keyword **`error`**.

The **`while IFS= read -r LINE`** loop reads the file line by line.

* `read -r` prevents backslashes from being interpreted as escape characters.

* `IFS=` temporarily sets the *Internal Field Separator* to an empty string, which means leading spaces are preserved.

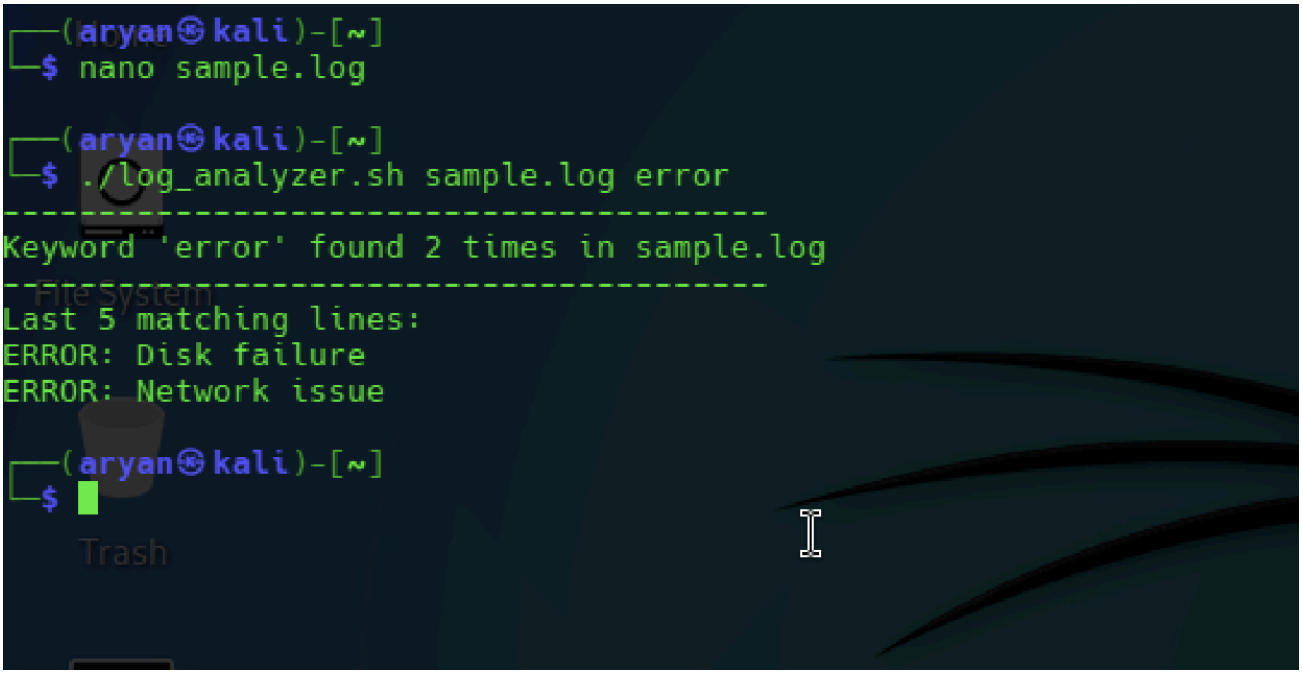
Inside the loop, the script searches for the keyword in the line and increments the count with **`COUNT=$((COUNT + 1))`**. This is called *arithmetic expansion* in Bash, which means you can perform arithmetic operations directly in the shell.

The loop also uses *input redirection* **`< "$LOGFILE"`** instead of using *cat*. This is more efficient because:

* It does not fork an extra process.

* It directly sends the file to the loop.

* It is generally recommended in shell scripting best practices.



Script 5 — The Open Source Manifesto Generator

Concepts: read for user input, string concatenation, writing to file with > and >>, date command, alias comment.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Addepalli Dhanush

# Alias example (optional use in terminal)
# alias manifesto="bash manifesto_generator.sh"

echo "Answer three questions to generate your manifesto."
echo ""

read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose and write manifesto
echo "----------------------------------------" > "$OUTPUT"
echo "Open Source Manifesto — $DATE" >> "$OUTPUT"
echo "----------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "I believe in the power of open source. Using tools like $TOOL every day
reminds me that technology should be accessible to all." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM — the ability to learn, modify, and share
without restrictions." >> "$OUTPUT"
```

```
echo "If given the opportunity, I would build $BUILD and share it with the world,
contributing to a community driven by collaboration and transparency." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "— $(whoami)" >> "$OUTPUT"

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat "$OUTPUT"
```

This script is designed to create a personal open source manifesto by prompting the user to input their responses, which are then written to a file in a formatted manner.

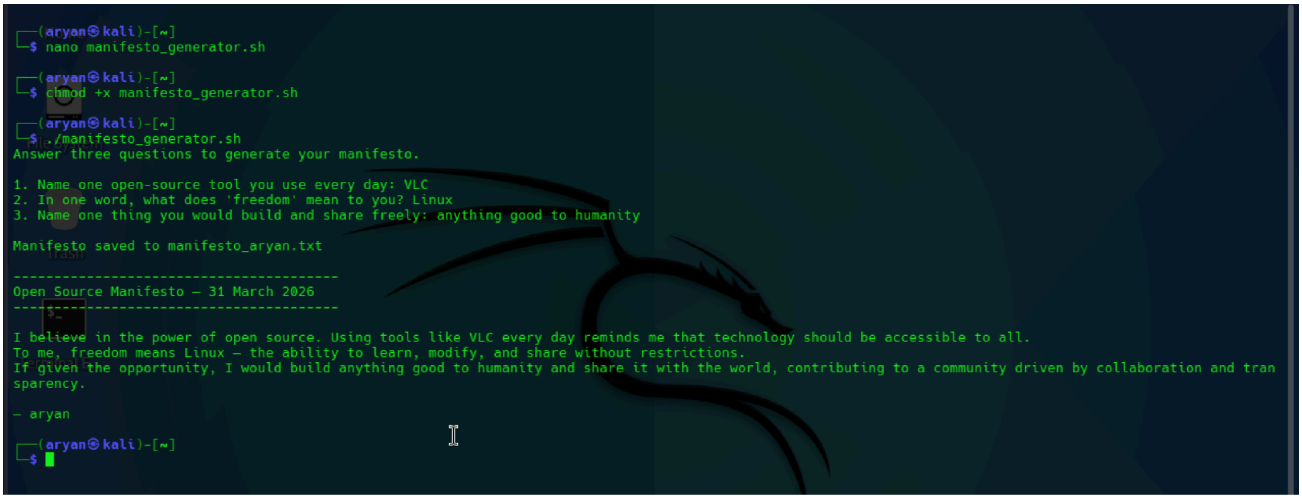
The user input is taken using the **`read -p`** command. This command is used to ask the user to input something, and the `-p` option is used to display the prompt message on the same line, so the user knows what they are expected to input. This input is then assigned to variables like `TOOL`, `FREEDOM`, `BUILD`, etc.

The text is formatted using the process of **string concatenation**, where the text is concatenated using the `echo` command. Each line is designed to add relevant sentences using the user's input, gradually building up the text to create a full paragraph, which is then written to the output file (for example, `manifesto_username.txt`).

The operators **`>` and `>>`** are used to write the text to the file:

* `>` is used to **create a new file or overwrite** the existing one (for the first line, where we want to start fresh).
* `>>` is used to **append** the text to the existing file (for the subsequent lines, where we do not want to overwrite the existing text).

> The **`date`** command is used with the option `'+%d %B %Y'`, which shows the date in a more readable format (for example, `31 March 2026`).
>
> The line that is commented out, which shows the **alias**, is where the user is shown how they could use the alias feature of the Bash shell. This feature is useful because the user could create their own alias using the alias command, which would enable the user to simply type `manifesto` at the terminal to execute the script.

> 
>

References

1. GNU Project — The Free Software Definition. gnu.org/philosophy/free-sw.html

2. Open Source Initiative — The Open Source Definition. opensource.org/osd

3. Eric S. Raymond — The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar

4. The Linux Command Line by William Shotts — freely available at linuxcommand.org

5. GNU Bash Manual: gnu.org/software/bash/manual

6. Linux man pages: man.cx (online) or type 'man ' in your terminal

7. SPDX License List: spdx.org/licenses

8. VLC legal page: www.videolan.org/legal.html